giga

GIGA PRODUCT DESIGN SYSTEM

One design system, better Giga products.

How Giga is moving from Carbon to its own shadcn-based design system — and what that unlocks.

— 01 · CONTEXT

Where we started: Carbon

Giga's products were built on IBM's **Carbon Design System** — fully for GigaMaps, only partly for GigaMeter.

GigaMaps

Carbon

Built on Carbon since integration started — components, tokens and patterns all inherited from IBM's system.

Good · Moderate · Bad · Unknown

Fully Carbon

GigaMeter

Partly Carbon

Some pieces Carbon, others its own — two apps under one brand, but two different behaviors.

↓ **48** Mbps DOWNLOAD

↑ **12** Mbps UPLOAD

28 ms LATENCY

Mixed foundations

— 02 · WHY SHADCN

Why shadcn/ui over Carbon

	Carbon	Giga shadcn
Flexibility	Opinionated — themed within Carbon's constraints.	Fully customizable, lives in our own repo.
Look & feel	IBM's visual language.	Cleaner, more modern — tailored to Giga.
Tokens	Tied to Carbon's theming system.	Better documentation & management.

— 03 · MIGRATION

We did not start from zero.

Existing foundations carried across — the migration reshaped them into one shared system.

1

Map the foundations

Tokens and colours collected across every Giga product into a single source of truth.

COLLECT

2

Tailor shadcn

Giga guidelines applied so components speak Giga by default — type, colour, radius, motion.

THEME

3

Roll out per product

GigaMaps first (health build), then GigaMeter — new components land already aligned.

SHIP

04 · PRODUCT ROLLOUT

Product rollout

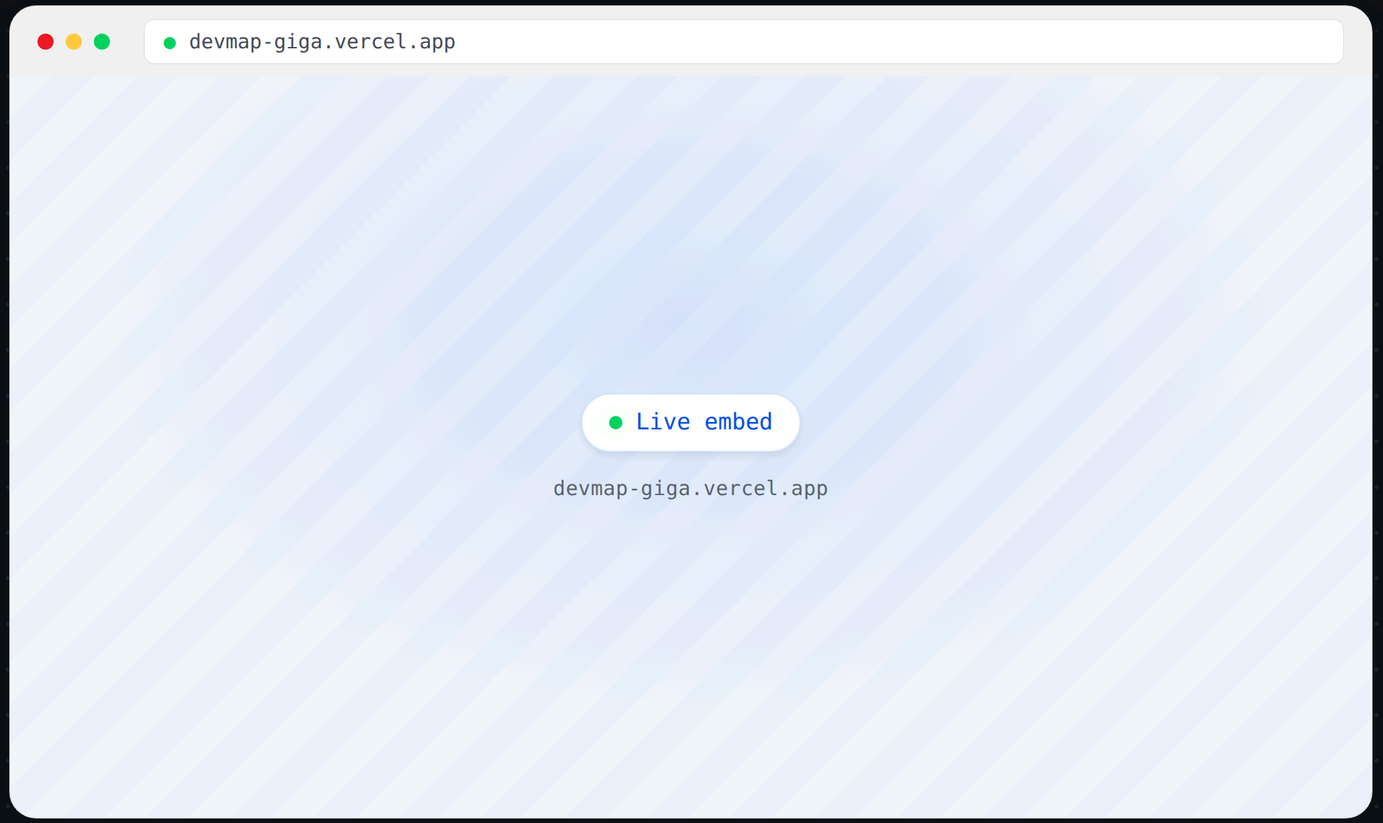
Where the new system meets real products — live, in production, in front of schools.

1 · GIGAMAPS · Live demo · Mario

GigaMaps

The school connectivity map. The **health build** opened the door for the new design system.

- Migration **started here**, inside the ongoing health phase.
- Every new component built to **fit the new system**.
- A **dedicated dev environment** for prototyping.



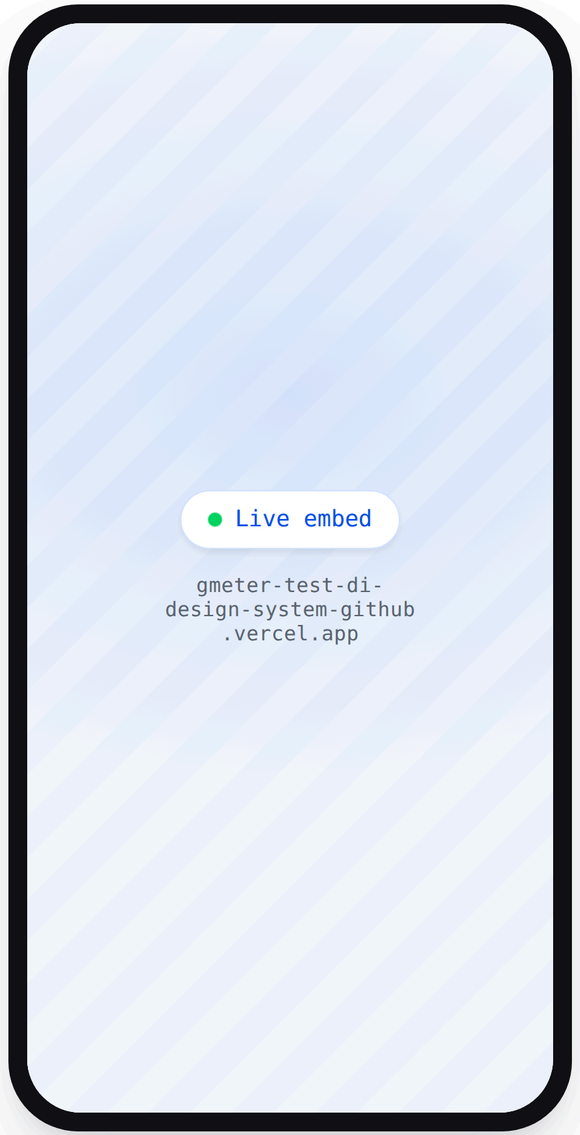
2 · GIGAMETER

Live demo · Jamil

Giga**Meter**

Measures real school connectivity — **download, upload, latency** — reported to the Giga platform.

- Aligning the UI to the new system **piece by piece**.
- New features prototyped **directly on new components**.
- Example: the **test-protocol prototype** from a previous demo.



GMETER-TEST-DI-DESIGN-SYSTEM-GITHUB.VERCEL.APP

— 3 · GIGA-DESIGN-SYSTEM

One repo, three pillars

GIGA-DESIGN-SYSTEM

```
app/ # showcase & docs
  components/
    ui/ # shadcn primitives
    giga/ # Giga composites
  lib/connectivity.ts ◆
  public/logos/
  skill/ # packaged agent skill
  registry.json
  CLAUDE.md ◆
```

Semantic design tokens

Colours, typography (Manrope / Open Sans), connectivity status colours, radii and motion.

Primitives on shadcn/ui

Themed primitives plus Giga composites — KPI tiles, connectivity legend, map panels, status markers.

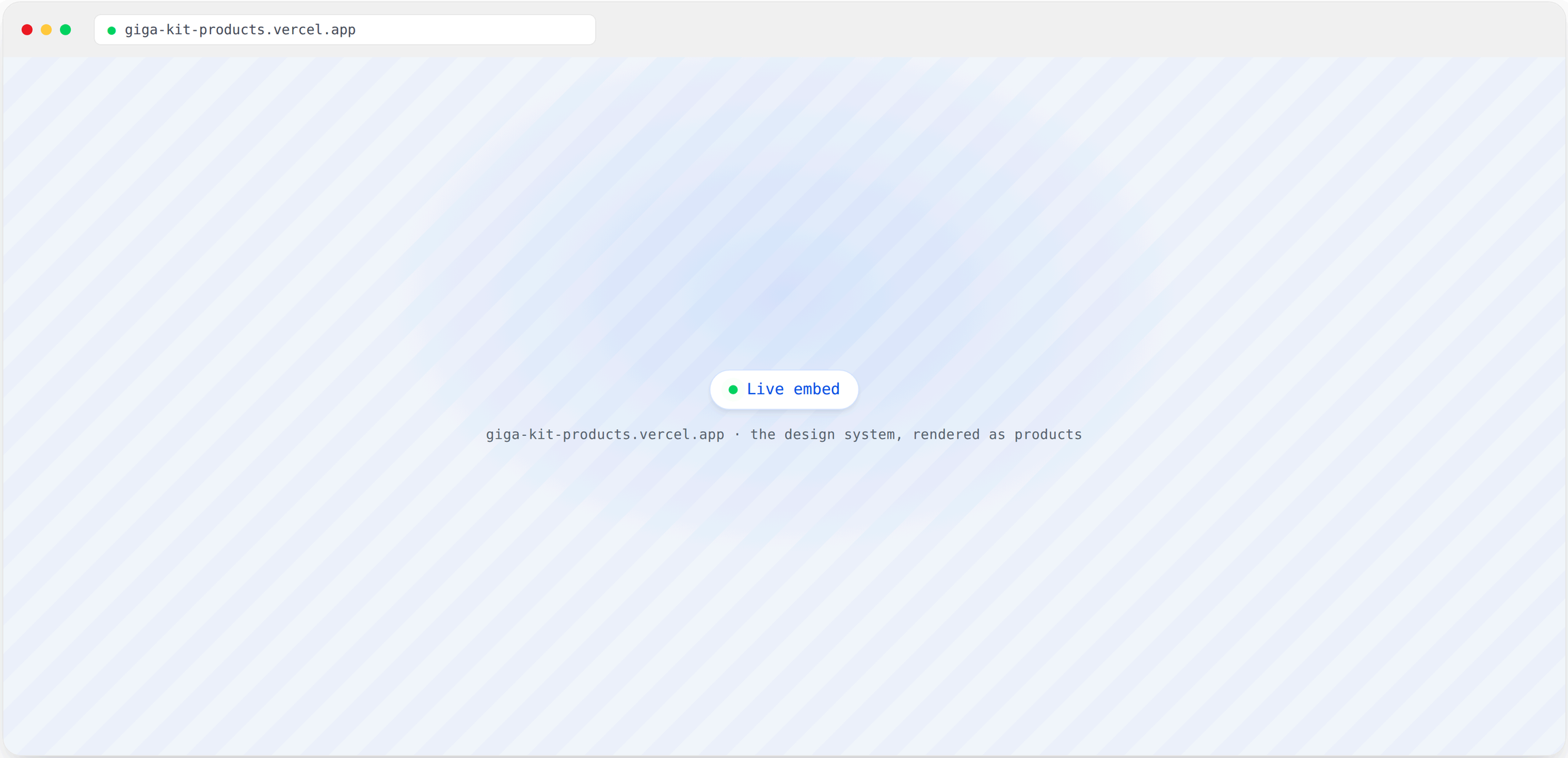
Agent baseline

CLAUDE.md + a packaged skill teaching Claude Code the tokens and components.

REPO OUTPUT

Live

giga-kit-products.vercel.app



— 05 · USE CASES

What you can do with it, today

Prototype product features

For GigaMaps and GigaMeter, right inside their own dev environments.

GIGAMAPS · GIGAMETER

Create new interfaces

Country insights, internal dashboards — Giga-styled by default.

DASHBOARDS · INSIGHTS

Ensure better handoff

More realistic, interactive handoff for vendors and stakeholders.

VENDORS · STAKEHOLDERS

06 · THE OUTPUT

From idea to Giga-branded prototype

1

Reference the repo

Clone it, or just point Claude Code at it.

2

Describe your feature

Plain language — the agent baseline picks the right primitives.

3

Get a working prototype

Right blue, right type, right components — by default.

EXAMPLE PROMPT

```
Build a dashboard for school connectivity alerts in Kenya
```

giga

Thank you.

One design system, better Giga products — from Carbon to a shadcn-based system that speaks Giga by default.